

Python Decorators

Decorators are one of the most powerful features in Python. They allow you to modify or extend the behavior of a function or class without changing its original code.

or

A decorator is a function that accepts another function as an argument, adds some functionality, and returns a new function.

Example : Pizza + Extra Cheese = Decorated Pizza

Similarly:

function + extra functionality = decorated function

Why Do We Need Decorators?

Imagine you have many functions and want to:

- ✧ Log function calls
- ✧ Check user authentication
- ✧ Measure execution time
- ✧ Validate input
- ✧ Handle exceptions

Instead of writing the same code repeatedly, decorators help reuse that logic.

1. Functions are First-Class Objects :

First-class objects are entities that can be created at runtime, assigned to variables, passed as arguments, and returned from functions.

Why Do We Need ?

Decorators work because Python treats functions as objects. Without first class function as objects, decorators would not exist.

Assign Function to Variable:

```
def greet():  
    print("Hello")  
x = greet  
x()
```

Output :

hello

Function Passed as Argument :

```
def greet():  
    print("Hello")  
def execute(func):  
    func()  
execute(greet)
```

Output:

Hello

This ability is the foundation of decorators.

2. Higher-Order Function :

A function that accepts another function as an argument or returns a function is called a higher-order function.

Example:

```
def greet():  
    print("Hello")  
def execute(func):  
    func()  
execute(greet)
```

Output:

Hello

3. Nested Function :

A function defined inside another function is called a nested function or inner function.

Why Do We Need?

Decorators Wrapper functions are nested functions.

Example:

```
def outer():  
    def inner():  
        print("Inner Function")  
    inner()  
outer()
```

Output:

Inner function

4. Returning Functions :

Returning a function means a function sends another function object back to the caller instead of executing it immediately.

Example :

```
def outer():  
    def inner():  
        print("Hello")  
    return inner  
result = outer()  
result()
```

Output:

Hello from Inner

5. Building First Decorator :

A decorator wraps another function.

Syntax :

```
def decorator(func):  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
    return wrapper
```

Example :

```
def greet():  
    print("Hello")  
greet = decorator(greet)  
greet()
```

Output:

```
Before  
Hello  
After
```

6. @ Symbol :

@ is the shortcut syntax for applying decorators in a clean & readable way.

Why do we need?

Makes code cleaner and readable

Instead of :

```
greet = decorators(greet)
```

Use:

```
@decorator  
def greet():  
    print("Hello")
```

7. Wrapper Function :

The inner function used in decorators to add additional functionality around the original function is called a wrapper function.

or

A wrapper function surrounds another function and executes code before or after the original function call.

Why do we need?

Wrapper controls execution before & after the original function.

Example :

```
def wrapper():  
    print("Before")  
    func()  
    print("After")
```

Wrapper acts as a middleman.

8. *args :

*args collects multiple positional arguments into a tuple.

Why do we need?

Decorator should work with any function.

Without *args:

```
def wapper():
```

It works only for no argument functions

Example:

```
def add(*args):
```

```
    print(args)
```

```
add(10,20,30)
```

Output:

```
(10,20,30)
```

9. **kwargs :

**kwargs collects multiple keyword arguments into a dictionary.

Example:

```
def info(**kwargs):
```

```
    print(kwargs)
```

```
info(name = "john",age=25)
```

Output :

```
{'name':'john','age':25}
```

10. Returning Values in Decorators :

A decorator should return the value returned by the original function.

- When a function returns something the decorators must capture that value and return it back to the caller.

Example :

Decorator:

```
def my_decorator(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        result = func(*args, **kwargs)
```

```
        return result
```

```
    return wrapper
```

Usage:

```
@my_decorator
```

```
def square(n):
```

```
    return n*n
```

```
print(square(5))
```

Output:

```
25
```

11. Closure :

A closure is a nested function that retains access to variables from its enclosing scope even after the outer function has completed execution.

Example :

```
def outer():
```

```
    x = 10
```

```
def inner():
    print(x)
    return inner
```

Usage:

```
f = outer()
f()
```

Output:

```
10
```

12. **functools.wraps()** :

`functools.wraps()` is a helper decorator used to copy metadata from the original function to the wrapper function.

What Is Metadata?

Metadata means information about a function.

Example:

```
def greet():
    """Greeting Function"""
```

Metadata:

```
greet.__name__
greet.__doc__
greet.__module__
```

Problem Without wraps()

Decorator:

```
def decorator(func):
    def wrapper():
        return func()
    return wrapper
```

Usage:

```
@decorator
def greet():
    pass
```

Now:

```
print(greet.__name__)
```

Output:

```
wrapper
```

Why?

After decoration:

```
greet = wrapper
```

Python sees wrapper instead of greet

Solution :

```
from functools import wraps
def decorator(func):
    @wraps(func)
```

```
def wrapper():  
    return func()  
return wrapper
```

Now:

```
print(greet.__name__)
```

Output:

```
greet
```

Why Do We Use wraps()?

Frameworks like Django, Flask, FastAPI, testing tools, documentation generators, and debuggers depend on metadata.

Without wraps(), they may behave incorrectly.

13. Logging Decorator :

A logging decorator records information about function execution for monitoring, debugging, and auditing purposes.

What Is Logging?

Logging means keeping a record of events.

Example:

```
User logged in  
Money transferred  
API called  
File deleted
```

Why Logging Is Needed

Suppose an application crashes.

Without logs:

Nobody knows what happened.

With logs:

```
Function Started  
Function Failed  
Error Occurred  
Developers can investigate.
```

Example :

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print(f"Calling {func.__name__}")  
        result = func(*args, **kwargs)  
        print(f"Finished {func.__name__}")  
        return result  
    return wrapper
```

Usage

```
@logger  
def add(a,b):  
    return a+b  
add(10,20)
```

Output:

Calling add
Finished add

14. Timing Decorator :

A timing decorator calculates and displays the execution duration of a function to help optimize performance.

Why Is It Needed?

Programs can become slow.

Developers need to identify:

Which function is causing delays?

Example :

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("Time:", end-start)
        return result
    return wrapper
```

Usage

```
@timer
def process():
    time.sleep(2)
process()
```

Output:

Time: 2.0

16. Authentication Decorator :

An authentication decorator verifies whether a user is authorized before allowing access to a function.

Why Is It Needed?

Sensitive resources should not be accessible to everyone.

Example

```
def login_required(func):
    def wrapper():
        logged_in = True
        if logged_in:
            return func()
        else:
            print("Access Denied")
    return wrapper
```

Usage

```
@login_required
def dashboard():
    print("Welcome User")
dashboard()
```

15. Multiple Decorators :

Multiple decorators allow several independent functionalities to be layered around a single function.

Example

```
@decorator1
@decorator2
def greet():
    print("Hello")
```

Why Use Multiple Decorators?

Suppose you want:

Authentication

Logging

Timing

all at once.

Instead of one huge decorator:

```
@login_required
@logger
@timer
def transfer_money():
    pass
```

17. Parameterized Decorators :

A parameterized decorator accepts configuration arguments that control how the decorator behaves.

Why?

Normal decorators:

```
@decorator
```

Parameterized decorators:

```
@decorator(value)
```

allow customization.

Example

```
def repeat(times):
    def decorator(func):
        def wrapper():
            for i in range(times):
                func()
        return wrapper
    return decorator
```

Usage


```
@repeat(3)
def greet():
    print("Hello")
```

Output:

```
Hello
Hello
Hello
```

18. Class Decorators :

A class decorator dynamically modifies or extends a class without changing its original source code.

Example

```
def add_method(cls):
    def display(self):
        print("Added Dynamically")
    cls.display = display
    return cls
```

Usage:

```
@add_method
class Student:
    pass
```

Now:

```
s = Student()
s.display()
```

Output:

```
Added Dynamically
```

Why?

Useful in frameworks that automatically register classes or inject functionality.

19. Built-in Decorators :

These are decorators already provided by Python.

❖ @staticmethod :

What Is It?

A method that belongs to the class but does not need: self or cls

Example

```
class Math:
    @staticmethod
    def add(a,b):
        return a+b
```

Why?

Addition doesn't depend on object data.

❖ @classmethod :

What Is It?

A method that works with class-level data and receives the class (cls) as its first parameter.

Example

```
class Student:  
    count = 100  
    @classmethod  
    def show_count(cls):  
        print(cls.count)
```

Why?

All objects share the same class variable.

❖ @property :

What Is It?

Converts a method into an attribute.

Example

```
class Person:  
    @property  
    def name(self):  
        return "John"
```

Usage:

```
p = Person()  
print(p.name)
```

instead of:

```
print(p.name())
```

Why?

Makes code cleaner and hides implementation details.